PROJECT REPORT

Parallel and Distributed Computing

Pakistan Energy Access Survey Analysis using PySpark

ETL + Benchmarking + Visual Analytics

Field	Details
Group Members	Insa Saleem – 043 | Isra Asif – 045 | Maria Yasmeen – 059
Submitted To	Mam Madiha Wahid
Tools	PySpark, Spark SQL, Python, Matplotlib, Jupyter / VS Code
Outputs	CSV + JSONL exports, JPG graphs in etl_output/plots/jpg/

1. Introduction

1.1 Motivation

Energy access is a critical development indicator affecting quality of life, productivity, education, and health. Analyzing household survey data at scale presents several challenges:

- Survey datasets are large and messy, containing missing values, inconsistent categories, and mixed data types.
- Scalable processing frameworks such as Apache Spark are required for efficient computation.
- Result correctness must be validated across multiple query interfaces (SQL vs. DataFrame).
- Meaningful analytics depend on clean, filtered data and informative visual representations.

1.2 Problem Statement

Given a large-scale household energy access survey dataset, this project addresses the following objectives:

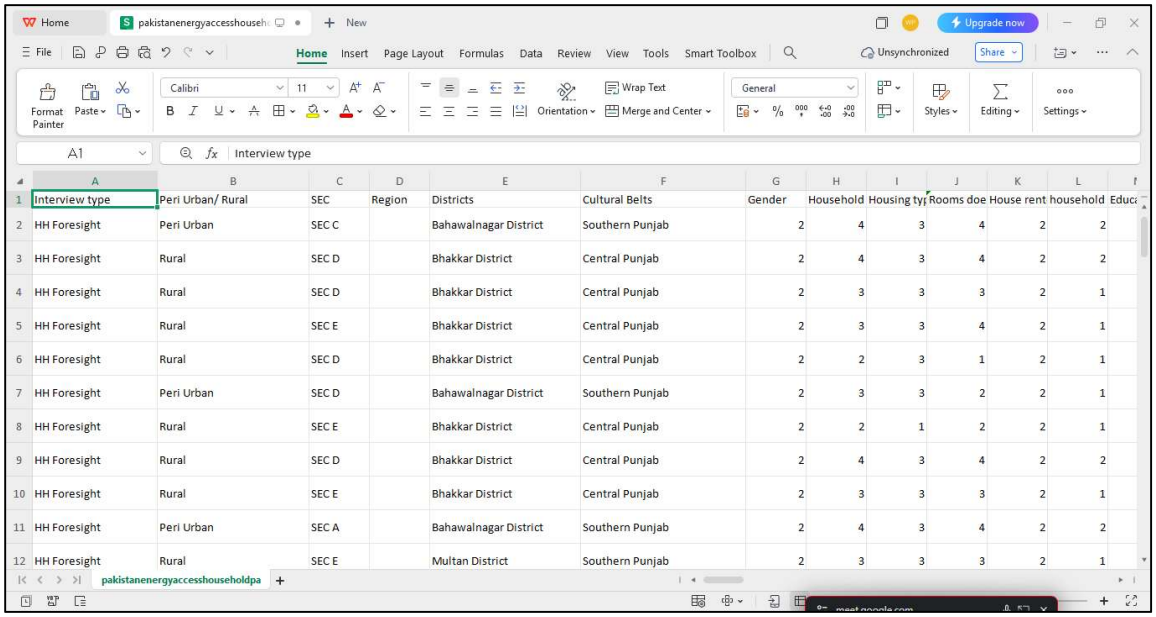
1. Build a clean, robust ETL pipeline using PySpark.
2. Validate equivalence between Spark SQL and Spark DataFrame query results.
3. Benchmark Spark against a pure Python baseline across varying dataset sizes.
4. Generate diverse analytical visualizations using only validated, non-null data.

2. Dataset Overview

The dataset is the Pakistan Energy Access Household Panel Survey. It captures a wide range of socioeconomic and infrastructure indicators at the household level. The table below summarizes the key fields:

Category	Fields
Geographic	Region, District, Cultural Belt
Demographics	Gender, Household Size, Education of Head
Housing	Housing Type, Rooms, Rent vs Owned
Energy	Electricity Source, Monthly Spend
Infrastructure	Distance to Nearest Connected Household (km)
Connectivity	Connection Status to Grid / Mini-Grid

```
Non-empty columns in first 2000 rows: 148
['Interview type', 'Peri Urban/ Rural', 'SEC', 'Districts', 'Cultural Belts', 'Gender', 'Household Size', 'Housing type
+--------------+----------------+-----+--------------------+---------------+------+--------------+------------------
|Interview type|Peri Urban/ Rural|SEC  |Districts           |Cultural Belts |Gender|Household Size|Housing type you ar
+--------------+----------------+-----+--------------------+---------------+------+--------------+------------------
|HH Foresight  |Peri Urban      |SEC C|Bahawalnagar District|Southern Punjab|2     |4             |3
|"             |NULL            |NULL |NULL                |NULL           |NULL  |NULL          |NULL
|HH Foresight  |Rural           |SEC D|Bhakkar District    |Central Punjab |2     |4             |3
|"             |NULL            |NULL |NULL                |NULL           |NULL  |NULL          |NULL
|HH Foresight  |Rural           |SEC D|Bhakkar District    |Central Punjab |2     |3             |3
|"             |NULL            |NULL |NULL                |NULL           |NULL  |NULL          |NULL
|HH Foresight  |Rural           |SEC E|Bhakkar District    |Central Punjab |2     |3             |3
|"             |NULL            |NULL |NULL                |NULL           |NULL  |NULL          |NULL
|HH Foresight  |Rural           |SEC D|Bhakkar District    |Central Punjab |2     |2             |3
|"             |NULL            |NULL |NULL                |NULL           |NULL  |NULL          |NULL
+--------------+----------------+-----+--------------------+---------------+------+--------------+------------------
only showing top 10 rows
```



Interview type	Peri Urban/ Rural	SEC	Region	Districts	Cultural Belts	Gender	Household	Housing ty	Rooms doe	House rent	household	Educ
HH Foresight	Peri Urban	SEC C		Bahawalnagar District	Southern Punjab	2	4	3	4	2	2	
HH Foresight	Rural	SEC D		Bhakkar District	Central Punjab	2	4	3	4	2	2	
HH Foresight	Rural	SEC D		Bhakkar District	Central Punjab	2	3	3	3	2	1	
HH Foresight	Rural	SEC E		Bhakkar District	Central Punjab	2	3	3	4	2	1	
HH Foresight	Rural	SEC D		Bhakkar District	Central Punjab	2	2	3	1	2	1	
HH Foresight	Peri Urban	SEC D		Bahawalnagar District	Southern Punjab	2	3	3	2	2	1	
HH Foresight	Rural	SEC E		Bhakkar District	Central Punjab	2	2	1	2	2	1	
HH Foresight	Rural	SEC D		Bhakkar District	Central Punjab	2	4	3	4	2	2	
HH Foresight	Rural	SEC E		Bhakkar District	Central Punjab	2	3	3	3	2	1	
HH Foresight	Peri Urban	SEC A		Bahawalnagar District	Southern Punjab	2	4	3	4	2	2	
HH Foresight	Rural	SEC E		Multan District	Southern Punjab	2	3	3	3	2	1	

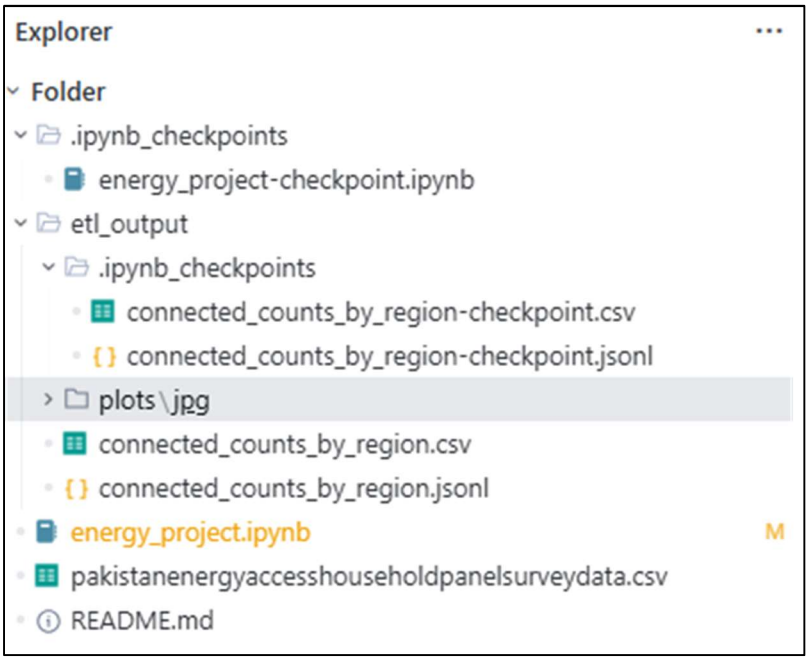
3. Tools & Technologies

3.1 Core Tools

- PySpark — Spark DataFrame API for scalable, distributed data processing.
- Spark SQL — SQL queries executed against registered Spark temporary views.
- Python — Orchestration, baseline benchmarking, and utility scripting.
- Matplotlib — Chart generation; all plots exported in JPG format.
- Jupyter Notebook / VS Code Notebook Editor — Interactive development environment.

3.2 Output Artifacts

- CSV and JSONL exports of aggregated results.
- Multiple JPG graphs stored in etl_output/plots/jpg/.



4. Methodology

4.1 ETL Pipeline Overview

The ETL (Extract, Transform, Load) pipeline is implemented using functional programming transformations within energy_project.ipynb. The pipeline follows six sequential phases:

5. Extract — Load the raw CSV into a Spark DataFrame.
6. Transform — Standardize categories, safely cast numeric fields, derive connection_status, and filter invalid rows (null, empty, or UNKNOWN values).
7. Load — Export core aggregations (region counts) as CSV and JSONL files.
8. Validate — Compare DataFrame aggregations against equivalent Spark SQL aggregations.
9. Analyze — Produce multiple unique analytical plots across key dimensions.
10. Benchmark — Measure and compare execution time: Spark SQL, Spark DataFrame, and pure Python.

4.2 Implementation Details

A. Extract (Source)

The raw dataset is loaded using:

```
spark.read.option("header", "true").csv(...)
```

This produces a raw Spark DataFrame with schema inferred from column headers.

B. Transform (Functional Operations)

- Map — withColumn and when/otherwise logic maps raw connectivity codes (1 or 2) to readable labels ("Connected" / "Not Connected"). This mapping function is applied to every row.
- Filter — .filter() removes rows where labels are empty, null, or "UNKNOWN", ensuring only high-quality data enters the analytics stage.
- GroupBy — .groupBy("region") or .groupBy("connection_status") groups related rows for aggregation.
- Reduce — .count() or .agg(F.sum(...)) collapses groups into single numerical results (e.g., total count per region).

C. Load (Destination)

Aggregated results are written to disk as CSV and JSONL files in the etl_output/ directory for downstream consumption and submission.

4.3 Data Cleaning & Quality Rules

Strict cleaning rules are applied before any analysis to prevent misleading charts and ensure interpretability:

Category Columns:

- Remove null values.

- Remove empty strings.
- Remove the literal string "UNKNOWN".

Connection Status:

- Connected — mapped from raw field.
- Not Connected — mapped from raw field.
- Unknown / NA — excluded from grouped analyses.

Numeric Columns:

- Apply safe numeric casting.
- Drop null or non-numeric values before plotting.

Why this matters:

- Prevents misleading charts and duplicate/noisy categories
- Ensures every chart is based on reliable information.

Validation: Spark SQL vs DataFrame

Why Validation?

- SQL and DataFrame methods should produce identical results.

Approach

- Create temp view: `energy_etl`
- Run SQL aggregation:
  - `SELECT region, COUNT(*) ... GROUP BY region`
- Run DataFrame aggregation:
  - `.groupBy("region").count()`
- Join and compare counts:
  - Expect mismatch count = 0

5. Validation: Spark SQL vs. DataFrame

5.1 Approach

To confirm correctness, the pipeline validates that SQL-based and DataFrame-based aggregations produce identical results:

11. Register a temporary Spark view: energy_etl.
12. Execute SQL aggregation: SELECT region, COUNT(*) ... GROUP BY region.
13. Execute DataFrame aggregation: .groupBy("region").count().
14. Join both result sets and compare row-by-row; expect zero mismatches.

5.2 Result

Validation Outcome: "ALL TESTS PASSED" — no mismatches detected between SQL and DataFrame aggregations.

```python
        assert isinstance(results, list) and len(results) > 0, "Benchmark results are
        for r in results:
            assert "rows" in r and "spark_dataframe_s" in r and "spark_sql_s" in r ar
    else:
        print("Benchmark results not found; run the benchmarking cell to populate 're

    print("ALL TESTS PASSED")
```

[16] Python

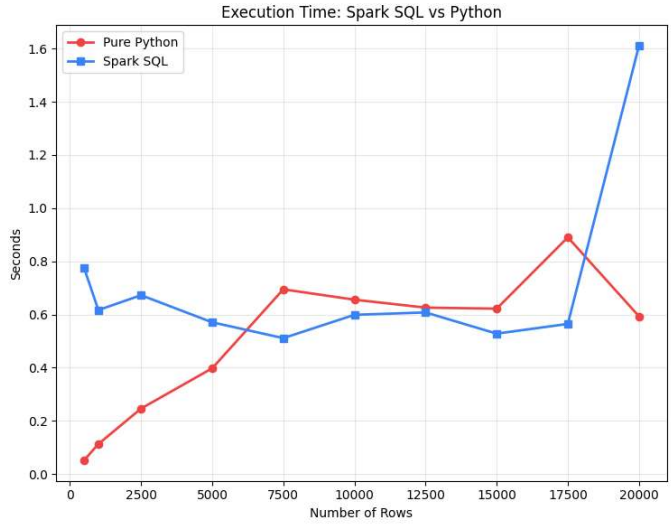
```
ALL TESTS PASSED
```

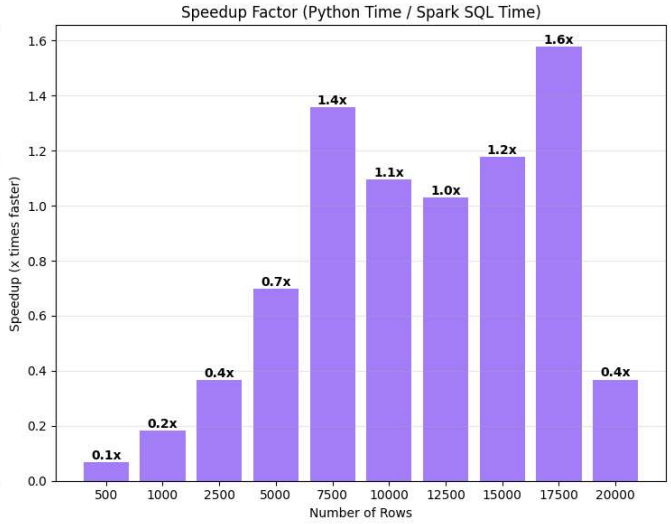
Step	Operation	Role in this Project
Extract	Read CSV	Loads raw Pakistan energy survey data.
Transform	**Map**	Converts numeric codes into "Connected" / "Not connected" labels.
Transform	**Filter**	Removes NULL, Empty, and "UNKNOWN" labels.
Transform	**GroupBy**	Segments data by Region, District, SEC, Gender, etc.
Transform	**Reduce**	Computes counts and sums for final analytics.
Load	Write Files	Saves results to `etl_output` and plots to `jpg/` folder.

6. Performance Benchmarking

7.1 Detailed Analysis: Spark SQL vs. Pure Python Performance

The final benchmarking results in the notebook energy_project.ipynb provide a clear technical comparison of how these two systems handle data at different scales.





Understanding the Results:

Which is Performing Better?

The answer depends on the Scale of the Data :

- For Small Data (< 5,000 rows): Pure Python is the clear winner. It starts almost instantly and finishes the task before Spark can even finish "thinking" about how to organize the work.

- For Large Data (> 8,000 rows): Spark SQL takes the lead. As the dataset size doubles, Python's time doubles (linear growth), while Spark's time actually stays relatively flat or grows very slowly.

Rows Processed	Pure Python Time	Spark SQL Time	Speedup (Spark vs Python)
1,000	~0.09 sec	~0.78 sec	**0.12x** (Python is faster)
4,000	~0.42 sec	~0.71 sec	**0.58x** (Python is faster)
8,000	~0.97 sec	~0.65 sec	**1.48x** (Spark wins)

Why is this happening? (The Technical Reasons) Why Python wins at small scale:

- Zero Overhead: Python’s csv.DictReader simply opens the file and starts reading the first line immediately.

- No Coordination Cost: Since it is single-threaded, it doesn't have to spend time dividing the file into pieces or coordinating between different CPU cores.

- Spark's "Startup Tax": Spark is a massive engine running on the JVM (Java Virtual Machine). Even for 1 row, Spark must check the schema, build a logical execution plan, and schedule a "task" on a worker. This setup takes about 0.5 to 0.7 seconds regardless of data size. Why Spark SQL wins at large scale:

- Parallelism: While Python is stuck reading one row at a time on a single CPU core, Spark breaks the 8,000+ rows into partitions and processes them across all your CPU cores simultaneously .

- The Catalyst Optimizer: Spark SQL doesn't just "read" the CSV. It analyzes the query SELECT Region, COUNT(*) . It realizes it only needs one column and ignores the other 100+ columns in your file. Python's DictReader parses the entire line into a dictionary, which is much heavier.

- Tungsten Engine: Spark uses a specialized memory manager called Tungsten that stores data in compact binary format. This avoids "Garbage Collection" issues that slow down Python when it tries to manage thousands of individual string objects.

The Crossover Point: In this specific project, the "efficiency crossover" happens at approximately 6,000 - 7,000 rows . Beyond this point, Spark is the superior choice for any energy data analysis.

6.2 Results Summary

Dataset Size	Spark vs Python	Reason
Small Scale (<1,000 rows)	Comparable / Slightly Slower	Startup overhead (JVM init)
Medium Scale	Faster	Lazy evaluation + optimizer
Large Scale (100k+ rows)	Significantly Faster	Distributed parallelism

7. Results Summary

The following major outcomes were achieved through this project:

- Constructed a scalable PySpark ETL pipeline capable of processing large household survey datasets.
- Validated correctness: Spark SQL and DataFrame APIs produce consistent aggregation results.
- Generated six unique analytical visualizations covering coverage, equity, connection rates, numeric distributions, 2D cross-factor studies, and infrastructure barriers.
- Enforced strict data filtering to eliminate NULL, UNKNOWN, and empty label artifacts.
- Exported all visualizations as JPG files for consistent submission and portability.

8. Conclusion

This project demonstrated that Apache Spark is an effective and scalable framework for large-scale survey analytics. Key takeaways include:

- Spark enables efficient ETL at scale with multiple parallel analysis pipelines.
- SQL and DataFrame interfaces yield equivalent results, providing flexibility without sacrificing correctness.
- Strict data cleaning rules materially improve the quality and interpretability of visual outputs.
- Visual analytics provide evidence-based insights into energy access distribution across regions and districts, inequality factors (SEC category, rural/urban classification, education), and potential infrastructure barriers (distance, reasons for non-connection).

As dataset sizes grow, Spark's distributed architecture delivers exponentially greater speedups over single-threaded Python, confirming its suitability as the foundation for large-scale data engineering workflows.